

# States, sampled edges and operators

These notes answer four comments from your earlier submissions. Each question is kept beside its explanation so you can connect the fix to what you were trying to do.

## Where this fits in your sheet

Entries 54, 74, 75, 77. Day 3, Day 5.

Entry numbers follow the tracker. Day labels in older filenames refer to when the notes were written; use the entry numbers to find the matching problem.

| Pages | What to read                                           |
|-------|--------------------------------------------------------|
| 2     | Entry 54: why the water-level controller needs history |
| 3     | Entry 74: comparing consecutive rising-edge samples    |
| 4     | Entry 75: making asynchronous reset wake the block     |
| 5-6   | Entry 77: reduction, bitwise and logical operators     |

Reviewed against the saved tracker and available code on 7 September 2026. Earlier submission dates inside these notes are historical records.

# Inputs are not a substitute for FSM state

The water sensors report the present level band. The state register stores any history that the outputs still need.

## Original submitted-code question

```
//localparam s2 // i dont need the check for specific cases cause they are already in the input
```

## Answer

The input bits and the FSM state have different jobs. A sensor pattern tells the circuit the level **now**. A **localparam** does not test that pattern; it gives a readable binary encoding to an internal state. The next-state logic must still interpret the sensor pattern, and the state register must retain any past information needed by a Moore output.

| Item                  | What it represents                         | Why it is needed                                          |
|-----------------------|--------------------------------------------|-----------------------------------------------------------|
| Sensor input          | Current water-level band                   | Chooses the next transition at this clock                 |
| localparam state name | A constant binary state encoding           | Makes the state machine readable and synthesizable        |
| state register        | Remembered operating condition/history     | Lets Moore outputs depend only on registered state        |
| case/if checks        | Mapping from sensor patterns to next state | Defines behavior for every valid input and a safe default |

## Why the present input can be insufficient

In this problem, dfr remembers whether the level last moved down or up while it stays in a middle band. The same sensor pattern can therefore need different dfr values. Separate states preserve that history.

```
localparam [2:0] FROM_ABOVE = 3'b001,
                FROM_BELOW = 3'b010,
                HIGH_LEVEL = 3'b100;

always @(*) begin
    next_state = state;           // safe default
    if (highest_sensor)
        next_state = HIGH_LEVEL;
    else if (middle_band)
        next_state = came_from_high ? FROM_ABOVE : FROM_BELOW;
    else
        next_state = FROM_BELOW;
end
```

## Important distinction

You may simplify checks for physically impossible sensor patterns if the prompt guarantees valid inputs. You may not remove a state when its remembered history changes an output.

# A synchronous edge is a difference between samples

The circuit observes one vector value at each rising clock edge and compares it with the value stored from the preceding edge.

## Original submitted-code question

```
// is it like 0 to 1 transition for any clock edge ? like in any 2 clock edges its that, transition
```

## Answer

Yes: for each bit, a positive edge is detected when the preceding rising-edge sample was 0 and the current rising-edge sample is 1. It is not a detector for every physical transition between clocks. A short pulse that rises and falls between two sampling edges can be missed.

| Previous sample | Current sample | in & ~previous | Meaning                     |
|-----------------|----------------|----------------|-----------------------------|
| 0               | 0              | 0              | no transition               |
| 0               | 1              | 1              | sampled positive edge       |
| 1               | 0              | 0              | falling edge, not requested |
| 1               | 1              | 0              | still high                  |

## Correct sequential implementation

```
reg [7:0] previous;

always @(posedge clk) begin
    previous <= in;
    pedge    <= in & ~previous;
end
```

## Why both assignments work in one block

Nonblocking assignments evaluate their right-hand sides using the values that existed when the block began. Therefore **pedge** uses the old **previous**, while **previous <= in** schedules the current sample for later. After the time step, the history register advances and the one-cycle pulse is available.

| Clock | old previous | sampled in | pedge computed | new previous |
|-------|--------------|------------|----------------|--------------|
| n-1   | 0            | 0          | 0              | 0            |
| n     | 0            | 1          | 1              | 1            |
| n+1   | 1            | 1          | 0              | 1            |

## Observed mistake

The unsuccessful expression **~in & previous** selects previous=1 and current=0, so it detects a sampled 1 -> 0 falling edge instead.

# An asynchronous reset must wake the block

This active-low reset must clear the state immediately when it falls, even if no rising clock edge occurs.

## Original submitted-code question

```
// why i have to write, asynchronous reset in the sensitivity list
```

## Answer

An always block runs only when an event in its event control occurs. If reset is asynchronous, reset assertion itself must be one of those events. The signal is named **aresetn**: the suffix **n** means active low, so the assertion event is **negedge aresetn**.

```
always @(posedge clk or negedge aresetn) begin
    if (!aresetn)
        state <= S_NONE;
    else
        state <= next_state;
end
```

| Form                              | When reset changes state       | Hardware described                           |
|-----------------------------------|--------------------------------|----------------------------------------------|
| @(posedge clk)                    | Only at a rising clock edge    | Synchronous reset if reset is checked inside |
| @(posedge clk or negedge aresetn) | Immediately when aresetn falls | Active-low asynchronous-reset flip-flops     |
| @(posedge clk or posedge areset)  | Immediately when areset rises  | Active-high asynchronous-reset flip-flops    |

## Event-by-event behavior

| Event                     | aresetn   | What the block does                      |
|---------------------------|-----------|------------------------------------------|
| negedge aresetn           | becomes 0 | Run immediately and load S_NONE          |
| posedge clk while reset=0 | still 0   | Run and keep S_NONE                      |
| posedge aresetn           | becomes 1 | No state update; wait for the next clock |
| next posedge clk          | 1         | Load next_state normally                 |

## Why not write bare aresetn?

The design needs a specific assertion edge that maps to a resettable flip-flop. **negedge** states both the polarity and the hardware event; omitting reset from the event control changes the behavior to synchronous.

Physical-design note: asynchronous assertion is often combined with synchronized deassertion to avoid recovery/removal timing problems. HDLBits is testing the RTL reset behavior shown above.

# Reduction, bitwise, and logical operators

The fastest way to choose correctly is to ask whether the desired result is one bit, one bit per vector position, or one Boolean truth result per whole operand.

## Original submitted-code question

```
// tell me difference b/w clearly b/w reduction and logical and bitwise
```

| Class     | Typical form        | Operation                                   | Result width |
|-----------|---------------------|---------------------------------------------|--------------|
| Reduction | &in,  in, ^in       | Combine every bit of one vector             | 1 bit        |
| Bitwise   | a & b, a   b, a ^ b | Combine aligned bit positions independently | Vector width |
| Logical   | a && b, a    b, !a  | Treat each whole operand as zero/nonzero    | 1 bit        |

## Worked example

Let **a = 4'b1010** and **b = 4'b1100**.

| Expression | Result  | Reason                              |
|------------|---------|-------------------------------------|
| &a         | 1'b0    | not every bit of a is 1             |
| a          | 1'b1    | at least one bit of a is 1          |
| ^a         | 1'b0    | a contains an even number of 1 bits |
| a & b      | 4'b1000 | AND each aligned pair of bits       |
| a && b     | 1'b1    | both complete vectors are nonzero   |
| !a         | 1'b0    | a is not the all-zero vector        |

## Correct answer for the four-input-gates problem

```
assign out_and = &in;
assign out_or  = |in;
assign out_xor = ^in;
```

## Why the submitted compound forms were wrong

An expression such as **out\_and &= in** is a compound read-modify-write assignment, conceptually **out\_and = out\_and & in**. It reads the previous output and does not reduce the bits of **in**. In combinational logic it also creates a self-dependency that can imply unwanted feedback.

### One-line selection rule

One vector in and one bit out: reduction. Two vectors and a vector out: bitwise. Whole conditions in if/while logic: logical.

# Revision checklist and verification record

Use this page as the quick recall sheet before resubmitting related HDLBits problems.

| Entry | Recall question                    | Correct recall                                                         |
|-------|------------------------------------|------------------------------------------------------------------------|
| 54    | Can inputs replace states?         | No. Inputs are present observations; states preserve required history. |
| 74    | What is a sampled positive edge?   | previous=0 and current=1 at consecutive rising-edge samples.           |
| 75    | Why is reset in the event control? | An asynchronous assertion must run the block without waiting for clk.  |
| 77    | Which operator collapses a vector? | A unary reduction operator such as &in,  in, or ^in.                   |

## Verification performed

- The original HDLBits pages and saved submitted-code comments were inspected in the signed-in Chrome session on 2026-09-02.
- The reusable Icarus Verilog self-check passed the corrected behavior for entries 74, 75, and 77.
- Entry 54 is a conceptual state-model answer; the generic snippet illustrates the separation of input decoding, state encoding, and remembered history rather than claiming a full replacement submission.
- The Attempt 2 tracker links these four entries directly to this focused PDF.

### Questions covered

The four questions are answered on pages 2-5. The water-level snippet shows the state/history idea; it is not a complete solution. The other three examples are covered by the saved Day 5 testbench.

## Original HDLBits references

[Entry 54 - exams/ece241\\_2013\\_q4](#)

[Entry 74 - edgedetect](#)

[Entry 75 - exams/ece241\\_2013\\_q8](#)

[Entry 77 - gates4](#)

### Submission reminder

A locally correct explanation or self-check is supporting evidence. HDLBits itself remains the authority for whether a submitted solution is accepted.